ATRIUM

Open Metaverse Platform

Project Summary

An open multiplayer 3D world platform built on glTF and WebSockets

A starter kit for a Metaverse for everyone, that nobody controls,
fully built on open standards and all open source.

glTF 2.0 | WebSockets | Three.js | Node.js | glTFTransform

Session 9 · March 2026

The Browser Model

Core philosophy

Navigate & Load

A world is a URL.

No account required to view.

No app to install.

Static First

glTF renders immediately.

Server connection is optional.

Graceful degradation built in.

Multiplayer Second

WebSocket layers on top.

World is visible before server connects.

Offline = scene still works.

> *"Atrium should feel like a browser, not a platform. You navigate to a world, it loads, you're there."*

Load Sequence:

Fetch atrium.json → Render glTF → Connect WS server → Sync world state

Architecture

Three layers

Content Layer

Standard glTF 2.0

- A world is a .gltf file
- Any glTF viewer can render it
- World metadata in extras.atrium
  - maxUsers, navigation info, server hints

Protocol Layer

SOP — Scene Object Protocol

- Lightweight WebSocket protocol
- 14 message types
- JSON Schema + Ajv validation
- Both client & server validated

Runtime Layer

Server + Client

- Node.js + glTF-Transform (server)
- Three.js + glTF-Transform (client)
- SOM — Scene Object Model API layer
- glTF is world state — SOM is the live API

atrium.json — World manifest: entry point for a deployed world. Contains glTF URL + WebSocket server URL.
Separates deployment config from content. Paths resolved relative to manifest file.

SOP — Scene Object Protocol

14 message types · JSON over WebSocket · Ajv validated

CLIENT → SERVER

hello	Initiate session, negotiate tick rate
ping	Latency check
send	Mutate a node field → broadcasts set
add	Add a new node to the world
remove	Remove a named node
view	Broadcast observer position & state

SERVER → CLIENT

hello	Session established
pong	Response to ping
tick	Heartbeat at negotiated interval
set	Authoritative world state update
add	Node added to world
remove	Node removed
join	Peer connected + position
leave	Peer disconnected
view	Peer observer state relay
error	Error with code + message

Key semantics: send/set mutations are echoed to sender as confirmation · view is NOT echoed back · view is fire-and-forget, last-write-wins · Avatar nodes live in the SOM scene graph — ephemeral per session, not persisted to world glTF

SOM — Scene Object Model

DOM-inspired API · glTF-Transform wrapper · symmetric server + client · 19 tests

Key Classes

Property accessors delegate to glTF-Transform · no redundant state

Class	Wraps	Role
SOMDocument	Document	Root entry point — analogue of browser document
SOMNode	Node	Scene graph node — translation, rotation, children
SOMMesh	Mesh	Primitives with PBR materials
SOMCamera	Camera	Perspective / orthographic + yfov
SOMAnimation	Animation	play() / stop() — server stubs, client live

setPath() — Dot-Bracket Deep Mutation

```
// Node-level property
som.setPath(node, "translation", [1, 0, 2]);

// Deep path — dot-bracket traversal
som.setPath(node, "mesh.primitives[0].material.baseColorFactor",
            [0, 1, 0, 1]);

// Camera and animation
som.setPath(node, "camera.yfov", 0.8);
som.setPath(node, "animation.loop", true);
```

Design Notes

- ► SOMDocument wraps a glTF-Transform Document — global som singleton is the browser document equivalent
- ► Simple values are properties, collections return arrays, methods only for operations with behavior
- ► JS get/set accessors delegate to glTF-Transform — no redundant state, Document is source of truth
- ► Symmetric — server and client both maintain a SOMDocument; DocumentView bridges SOM to Three.js on the client only
- ► glTF is world state — serializing the Document serializes the world, avatar nodes and all
- ► ingestNode(descriptor) — adds a full glTF node including geometry from a client add message

Key design: SOM node names map 1:1 to glTF JSON node names · composition over subclassing — SOM wraps glTF-Transform · setPath walks any dot-bracket path at runtime · send/set SOP mutations drive setPath on the server

What's Been Built

Session 9 complete — all tests passing

packages/protocol

- 14 SOP message schemas (JSON Schema draft-07)
- validate(direction, message) for client & server
- 41 passing tests

packages/server

- WebSocket server, session lifecycle, keepalive
- World state via glTF-Transform Document
- Avatar nodes in SOM — full connect/disconnect lifecycle
- view message relay · SOM dump on connect · 32 passing tests

tests/client/index.html

- Single-file, no build step
- Walk camera (WASD + mouse) default
- Peer avatars via SOM — capsule geometry, live position updates
- Loads glTF immediately — static first

tests/fixtures

- space.gltf — ground plane, crate, lamp
- space.atrium.json — world manifest
- PBR materials, embedded geometry
- extras.atrium — full NavigationInfo object

@atrium/som

- DOM-inspired API over glTF-Transform
- setPath() — dot-bracket deep mutation
- Symmetric — server and client both use SOM
- 19 passing tests

Repository

- pnpm monorepo
- ES modules, no TypeScript, no build
- Node.js v20 · node --test · 92 tests total
- github.com/tparisi/atrium

Workstreams Ahead

Design before code — three major threads

1 User Object Extensions

ATRIUM_user_object

- Single vendor extension for all UO types
- Self-describing property model with targets + onchange
- Graceful degradation — plain glTF for non-Atrium viewers
- Authoring → marketplace → instance pipeline
- Open: scene graph fragment instancing, script context,
- type definition location, authority model

Design in progress ✦

2 Avatar & Navigation

view message + NavigationInfo

✓ view carries: position, look, move, velocity, up

✓ Unit vectors for look/move/up — no Euler/quaternion

✓ up optional, defaults [0,1,0] — only sent for FLY/roll

- Dead reckoning on receiver, absolute position for correction

✓ NavigationInfo: mode (array), terrainFollowing, speed, updateRate

- Deferred: dead reckoning, send delta thresholds, FLY mode up

Complete ✓

3 Real Browser Client

packages/client v0.1

- URL bar — navigate to any atrium.json or .gltf
- No SOP messaging panel (dev tool only)
- Full first-person navigation + look/move/velocity broadcast
- Proper avatar rendering (not just capsules)
- World metadata display — name, user count
- Depends on: Avatar & Navigation design ✓

Design in progress ✦

User Object Extensions

ATRIUM_user_object — design brief

The Concept

- One vendor extension for all user-defined types
- Type name inside the extension identifies the object
- Single ATRIUM_user_object schema, stable versioning
- Geometry stays in plain glTF nodes — graceful degradation
- Non-Atrium viewers render the object as normal glTF
- extensionsUsed (not Required) — degradation expected

Property Model

name	Logical property name
type	boolean / string / number (drives editor UI)
value	Default (type def) or current (instance)
target	node + property — direct glTF mapping
onchange	Script handler — called on value change

Instance in glTF

```
{
  "name": "record-player-01",
  "children": [1, 2, 3],
  "extensions": {
    "ATRIUM_user_object": {
      "type": "record_player",
      "properties": {
        "playing": true,
        "track": "a-love-supreme",
        "volume": 0.6
      }
    }
  }
}
```

The Authoring Flow

- Author UO in Atrium editor → publish to marketplace
- User finds asset, drops into world via editor
- Editor merges subtree into world glTF
- Runtime: arrives as add SOP message
- All clients instance it — Atrium or not

Avatar & Navigation Design

view message protocol · NavigationInfo · design brief

view Message Fields

Complete snapshot · all values absolute · dead reckoning on receiver

Field	Type	Req	Description
position	[x, y, z]	Yes	Absolute world position
look	[x, y, z]	Yes	Forward unit vector — camera direction
move	[x, y, z]	Yes	Movement direction unit vector
velocity	number	Yes	Speed in meters per second
up	[x, y, z]	No	Up vector — omit in WALK, send for FLY/roll

NavigationInfo (extras.atrium)

```
"navigation": {
  "mode": ["WALK", "FLY", "ORBIT", "TELEPORT"],
  "terrainFollowing": true,
  "speed": { "default": 1.4, "min": 0.5, "max": 5.0 },
  "collision": { "enabled": false },
  "updateRate": { "positionInterval": 1000, "maxViewRate": 20 }
}
```

Field notes:

- mode — array: WALK / FLY / ORBIT / TELEPORT, first is default
- terrainFollowing — avatar stays on ground surface
- updateRate — positionInterval (ms) + maxViewRate (msg/s)
- speed — default/min/max in m/s (matches velocity field)
- collision — object or omit; proxy = named node name

Key Decisions & Deferred Items

What's locked in · what's still open

✓ Locked Decisions

- HTTP serving stays separate — WebSocket server only
- Avatar nodes live in the SOM — ephemeral per session, not persisted to world glTF
- view message — fire-and-forget, last-write-wins, not echoed
- Walk camera is the default mode (not Orbit)
- atrium.json manifest separates deployment config from content
- Static first, multiplayer second — glTF renders before server connects
- ATRIUM_user_object — one extension for all user-defined types
- Geometry in plain glTF nodes — graceful degradation always
- view fields: position + look + move + velocity + up (optional)
- NavigationInfo in extras.atrium — migrates to ATRIUM_world extension later

⟳ Deferred

Flaky disconnect test	Race condition in session.test.js #19
README / TESTING.md	Dev workflow documentation
glTF extension package	ATRIUM_world + ATRIUM_avatar schemas
packages/client promotion	When design is more settled
Script execution context	Sandbox, trust model, client vs server
Scene graph instancing	UO type → multiple instances in glTF
Type definition location	Inline vs URI reference
Collision implementation	Proxy mesh, physics material
Viewpoints	Named cameras, default entry point
glXF alignment	When Khronos spec stabilizes

ATRIUM

github.com/tparisi/atrium

How to Run

```
git clone https://github.com/tparisi/atrium.git
cd atrium && pnpm install

# Run tests
pnpm --filter @atrium/protocol test
pnpm --filter @atrium/server test

# Terminal 1 — world server
WORLD_PATH=tests/fixtures/space.gltf node packages/server/src/index.js

# Terminal 2 — HTTP server
npx serve -l 5173 tests/

# Open client
http://localhost:5173/client/index.html
```

Session 9 complete · 41 protocol · 19 SOM · 32 server · 92 total